

Pontificia Universidad Católica de Chile

Facultad de Ingeniería

DCCubos

Inteligencia Artificial

Segundo Semestre 2026

Albert Roca PAS816129

03/10/2026

Contents

1 DCCubos: clasificación de piezas **2**

1.1 Uso de IA y planteamiento . . . 2
1.2 Método común . . . 2
1.3 Track A — Imágenes . . . 3
1.3.1 A.a — Características visuales . . . 3
1.3.2 A.b — Efecto del escalado . . . 4
1.3.3 A.c — Partición y estrategia de validación . . . 5
1.3.4 A.d — Comparación de modelos . . . 5
1.3.5 A.e — Efecto de los hiperparámetros . . . 6
1.3.6 A.f — Mejor y peor configuración . . . 7
1.4 Track B — Ficha técnica . . . 7
1.4.1 B.a — Exploración de las variables . . . 7
1.4.2 B.b — Limpieza y preprocesamiento . . . 9
1.4.3 B.c — Selección de variables y análisis de `category` . . . 9
1.4.4 B.d — Comparación de modelos tabulares . . . 11
1.5 Track C — Combinación . . . 12
1.5.1 C.a — Unión y selección de características . . . 12
1.5.2 C.b — Comparación de modelos y de los tres tracks . . . 12
1.5.3 C.c — Ranking global por validación . . . 14
1.5.4 C.d — Matrices de confusión e interpretación . . . 14
1.6 Conclusiones . . . 16
1.7 Anexo de IA . . . 18

1 DCCubos: clasificación de piezas

1.1 Uso de IA y planteamiento

Para la realización de esta parte del trabajo se utilizó inteligencia artificial generativa como herramienta de apoyo en la planificación del análisis, revisión y documentación del código y la interpretación inicial de los resultados. Los resultados numéricos presentados en esta sección proceden, en todos los casos, de la ejecución del notebook entregado. La selección de modelos y las conclusiones finales se realizaron a partir de dichos resultados, evitando utilizar el conjunto de prueba para escoger hiperparámetros.

El objetivo del problema es clasificar imágenes correspondientes a ocho tipos de piezas LEGO:

- `3001 brick 2x4`
- `3003 brick 2x2`
- `3004 brick 1x2`
- `3005 brick 1x1`
- `3022 Plate 2x2`
- `3040 roof tile 1x2`
- `3062 Round Brick 1x1`
- `3069 Flat Tile 1x2`

Se estudiaron tres enfoques diferentes. El **Track A** utiliza únicamente información extraída de las imágenes; el **Track B** utiliza la información contenida en la ficha técnica; y el **Track C** combina ambas fuentes de información.

El criterio principal empleado para comparar configuraciones fue la *accuracy* media obtenida mediante validación cruzada. Como medida complementaria se utilizó el F1 macro, ya que el dataset está desbalanceado. El conjunto de test se mantuvo reservado hasta haber seleccionado las configuraciones mediante validación.

1.2 Método común

El conjunto de datos contiene un total de 2.400 observaciones, con 300 ejemplos de cada una de las ocho clases. La partición que elegí fue del 80 % para train y del 20 % para test. De esta forma, obtuve 1.920 observaciones de entrenamiento y 480 de prueba, con exactamente 240 y 60 ejemplos de cada clase, respectivamente.

Sobre las 1.920 observaciones de entrenamiento realicé una validación cruzada de cinco folds. Los Tracks A, B y C utilizan exactamente los mismos índices de entrenamiento, prueba y validación, lo que permite realizar una comparación directa entre ellos.

Todos los pasos que requieren aprender información de los datos, como el escalado, la imputación o la selección de variables, se ajustaron exclusivamente utilizando el conjunto de entrenamiento correspondiente a cada fold. Esto evita introducir información procedente de validación o test en el entrenamiento (*data leakage*).

El conjunto de prueba no lo toqué durante las pruebas de los hiperparámetros. Solo lo utilicé para saber el rendimiento del test sin que el modelo hubiera visto nunca esos datos.

1.3 Track A — Imágenes

1.3.1 A.a — Características visuales

Para representar cada imagen utilicé características HOG (*Histogram of Oriented Gradients*) junto con seis características geométricas obtenidas a partir de la segmentación de la pieza.

La Figura 1 muestra ejemplos de la segmentación utilizada y de la representación HOG obtenida. Se puede observar cómo la máscara separa la pieza del fondo y cómo HOG conserva principalmente información relacionada con los bordes y cambios de orientación de la pieza.

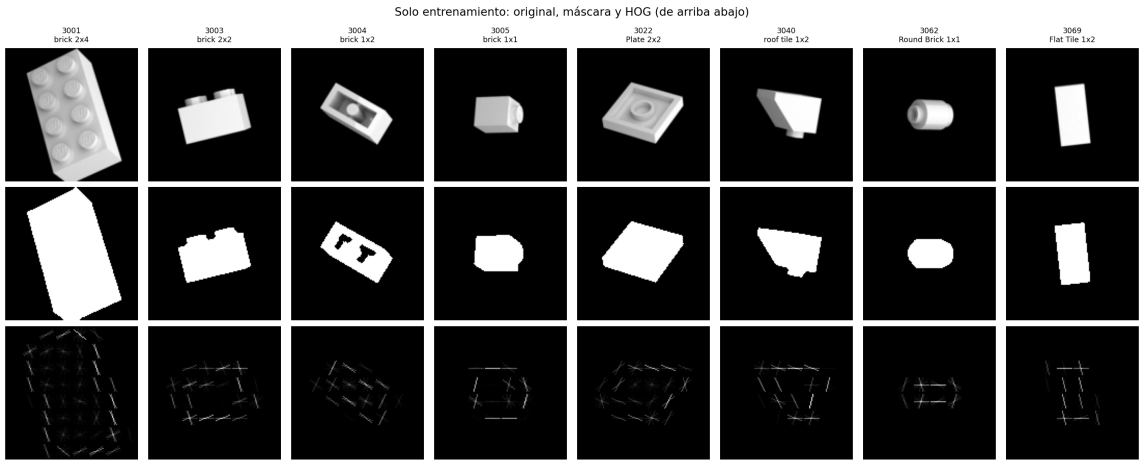


Figure 1: Ejemplos de segmentación y representación HOG de las piezas.

La configuración utilizada genera 1.764 componentes HOG. A estas se añadieron seis características geométricas:

- área relativa;
- relación entre anchura y altura;
- solidez;
- excentricidad;
- extensión;
- circularidad.

Por tanto, cada imagen queda inicialmente representada mediante 1.770 características.

Los gráficos de distribución muestran que algunas medidas geométricas contienen información discriminativa. Por ejemplo, la mediana del área relativa de la pieza `3001 brick 2x4` es aproximadamente 0,363, mientras que para `3062 Round Brick 1x1` es aproximadamente 0,062, por lo que ambas distribuciones aparecen claramente separadas.

Sin embargo, otras variables muestran un solapamiento mucho mayor. La excentricidad mediana de `3003 brick 2x2` es aproximadamente 0,536, frente a 0,530 para `3005 brick 1x1`, por lo que esta característica por sí sola prácticamente no permite distinguirlas. Algo similar ocurre con la solidez, cuyas medianas se encuentran alrededor de 0,95 para todas las clases.

La Figura 2 permite observar visualmente estas diferencias. Algunas clases se separan claramente en variables como el área relativa, mientras que en características como la excentricidad o la solidez existe bastante solapamiento entre varias piezas.

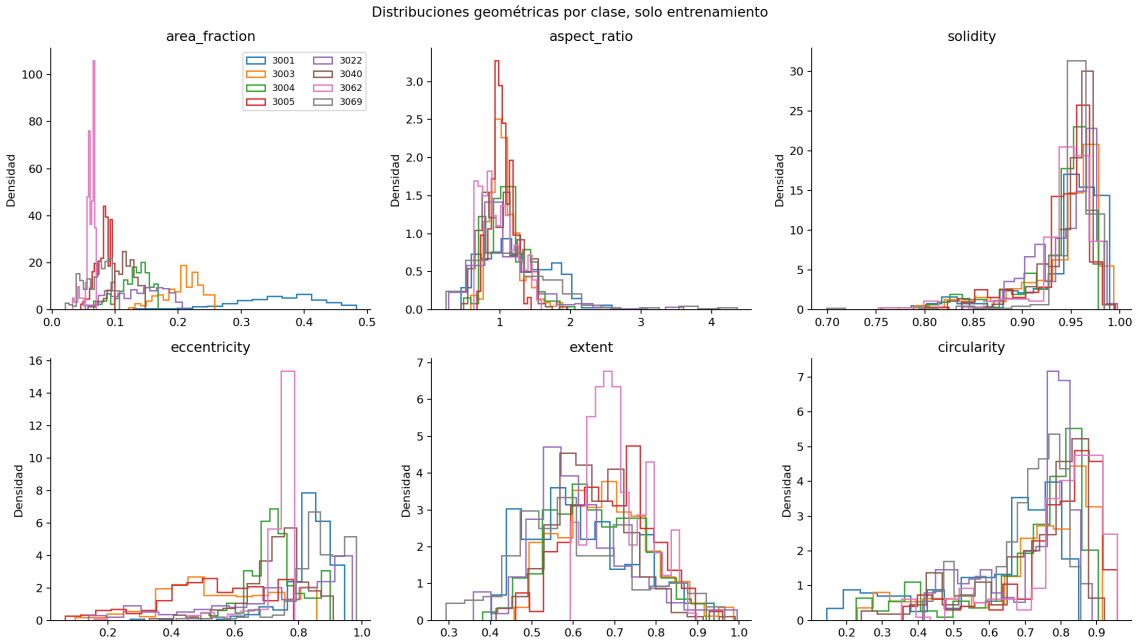


Figure 2: Distribuciones por clase de las características geométricas.

Estos resultados muestran que las variables geométricas aportan información global útil, pero no son suficientes para resolver por sí mismas el problema. HOG complementa estas características al conservar información local sobre bordes y estructura interna.

Conclusión. La combinación de HOG con variables geométricas permite representar simultáneamente información local y global de la pieza. Sin embargo, estas características siguen dependiendo de la orientación, perspectiva y calidad de la segmentación, por lo que no garantizan invariancia frente a cambios importantes en la imagen.

1.3.2 A.b — Efecto del escalado

Antes de entrenar los modelos apliqué a los datos el estandarizador `StandardScaler`, ajustándolo exclusivamente sobre los datos de entrenamiento de cada fold.

El escalado resulta especialmente importante para SVM y KNN. KNN toma decisiones basándose directamente en distancias entre observaciones, mientras que el comportamiento de SVM depende de las distancias y del margen generado en el espacio de características. Si algunas variables tienen escalas numéricas mucho mayores que otras, podrían opacar a otras variables.

En cambio, los árboles de decisión y Random Forest dividen el espacio mediante umbrales aplicados individualmente sobre cada variable. Un reescalado lineal positivo mantiene esencialmente el orden de los valores y, por tanto, tiene un efecto mucho menor sobre las particiones que pueden realizar estos modelos.

Conclusión. El escalado es necesario principalmente para garantizar una comparación adecuada en SVM y

KNN, mientras que su impacto teórico sobre los modelos basados en árboles a priori debería ser reducido.

1.3.3 A.c — Partición y estrategia de validación

La partición 80/20 permite mantener una cantidad suficientemente grande de observaciones para entrenamiento y validación, reservando al mismo tiempo 480 ejemplos completamente independientes para la evaluación final.

La estratificación garantiza que todas las clases mantengan la misma representación en ambos conjuntos: 240 ejemplos por clase en entrenamiento y 60 en prueba.

Dentro del conjunto de entrenamiento se utilizaron cinco folds compartidos entre todas las configuraciones. De esta forma, las diferencias observadas entre modelos no pueden atribuirse a que hayan sido evaluados utilizando particiones diferentes.

El conjunto de test únicamente se utilizó después de establecer el ranking de validación. Por tanto, las decisiones sobre modelos e hiperparámetros no dependen de su resultado final en prueba.

Conclusión. La estrategia empleada permite realizar una comparación coherente entre modelos y obtener posteriormente una estimación independiente de su capacidad de generalización. Sin embargo, esta evaluación mide el rendimiento sobre renders de estas ocho clases concretas y no permite asumir directamente el mismo rendimiento sobre fotografías tomadas en condiciones reales de producción.

1.3.4 A.d — Comparación de modelos

Se evaluaron cuatro familias de modelos: SVM, Random Forest, Decision Tree y KNN. Para cada una se probaron tres configuraciones diferentes.

Table 1: Resultados de validación del Track A.

Configuración	Accuracy CV	Desv. CV	Accuracy train
A_RandomForest_2	87,60 %	2,39 pp	100,00 %
A_RandomForest_3	87,45 %	2,51 pp	100,00 %
A_SVM_3	87,40 %	2,84 pp	99,95 %
A_SVM_2	83,49 %	2,74 pp	92,08 %
A_DecisionTree_2	78,44 %	3,19 pp	93,95 %
A_DecisionTree_3	77,66 %	3,22 pp	100,00 %
A_RandomForest_1	74,43 %	3,26 pp	83,97 %
A_KNN_1	74,32 %	2,01 pp	100,00 %
A_KNN_2	69,11 %	2,24 pp	100,00 %
A_SVM_1	68,80 %	1,80 pp	74,14 %
A_KNN_3	60,73 %	2,11 pp	100,00 %
A_DecisionTree_1	55,42 %	1,38 pp	56,07 %

La Figura 3 permite comparar de forma visual las doce configuraciones. Random Forest y las configura-

ciones más altas de SVM son las que consiguen mejores resultados, mientras que `A_DecisionTree_1` queda claramente por debajo del resto.

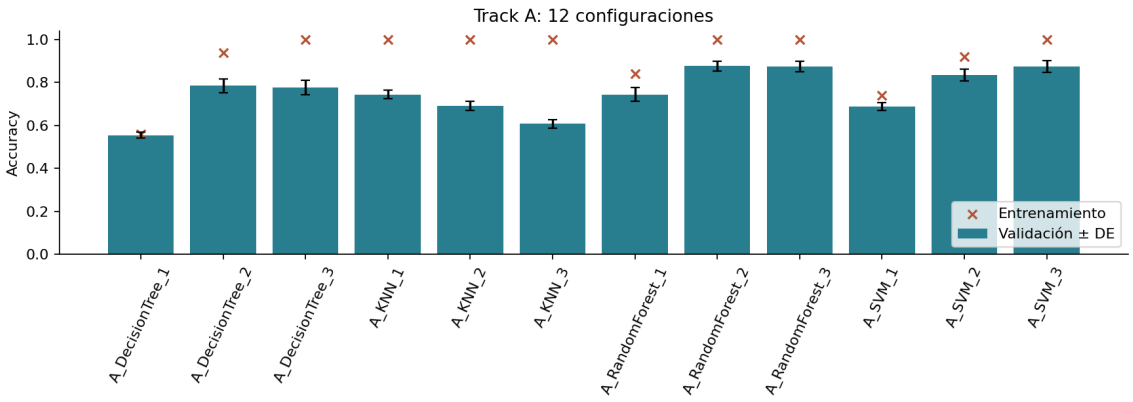


Figure 3: Resultados de validación de las doce configuraciones del Track A.

El mejor resultado de validación corresponde a `A_RandomForest_2`, con una accuracy media de 87,60 %. Al evaluarlo posteriormente sobre el conjunto de prueba alcanza un 90,83 % de accuracy y un F1 macro de 90,77 %, clasificando correctamente 436 de las 480 imágenes.

Si consideramos el mejor modelo de cada familia, los resultados de test fueron los siguientes:

Table 2: Modelos finalistas del Track A.

Modelo	**Accuracy CV**	**Accuracy test**
Random Forest	87,60 %	90,83 %
SVM	87,40 %	89,17 %
Decision Tree	78,44 %	80,42 %
KNN	74,32 %	76,25 %

Random Forest obtiene el mejor rendimiento global. El hecho de combinar las predicciones de numerosos árboles permite capturar interacciones complejas entre las características visuales y reducir parte de la variabilidad que tendría un único árbol.

No obstante, su accuracy de entrenamiento es del 100 %, considerablemente superior al 87,60 % de validación. Esto muestra que el modelo tiene capacidad suficiente para memorizar prácticamente el conjunto de entrenamiento y no mantiene esos resultados tan buenos en el test. En esta ejecución, sin embarg no hay una caída muy grande de rendimiento sobre test.

Conclusión. Random Forest es el modelo visual más competitivo de los evaluados, aunque SVM obtiene un rendimiento muy cercano. Por tanto, los resultados apoyan la elección de Random Forest para esta ejecución, pero no permiten afirmar que sea superior a SVM.

1.3.5 A.e — Efecto de los hiperparámetros

Los resultados muestran que la elección de hiperparámetros tiene un impacto considerable.

En SVM, aumentar el parámetro C desde 0,1 hasta 10 incrementa la accuracy de validación desde 68,80 % hasta 87,40 %. Un valor pequeño de C impone una regularización fuerte y permite un mayor número de errores para mantener una frontera más simple. En este problema, esa regularización resulta excesiva y conduce a underfitting.

En Random Forest, limitar la profundidad a 5 produce una accuracy de 74,43 %. Aumentarla hasta 15 eleva el resultado hasta 87,60 %. Sin embargo, eliminar completamente el límite de profundidad no mejora el rendimiento, que se sitúa en 87,45 %. Esto indica que aumentar la complejidad ayuda inicialmente, pero a partir de cierto punto deja de aportar capacidad de generalización.

En Decision Tree se observa un comportamiento similar. La profundidad 3 obtiene únicamente 55,42 %, mientras que una profundidad máxima de 8 alcanza 78,44 %. El árbol sin límite obtiene 77,66 %, ligeramente peor.

Finalmente, en KNN ocurre el comportamiento contrario: pasar de 3 a 15 vecinos reduce el resultado desde 74,32 % hasta 60,73 %. En este caso, utilizar demasiados vecinos suaviza excesivamente las fronteras entre clases.

Conclusión. Los resultados muestran claramente el compromiso entre complejidad y generalización. Una mayor complejidad mejora algunos modelos cuando parten de una configuración demasiado restrictiva, pero no existe una relación monotónica por la cual un modelo más complejo sea necesariamente mejor.

1.3.6 A.f — Mejor y peor configuración

Según el criterio fijado previamente de accuracy media en validación cruzada, la mejor configuración del Track A es `A_RandomForest_2`, con un 87,60 %.

La peor configuración es `A_DecisionTree_1`, que utiliza una profundidad máxima de 3 y obtiene únicamente un 55,42 % de accuracy de validación.

Además, su accuracy de entrenamiento es de 56,07 %, prácticamente igual a la obtenida mediante validación. Esto indica que el problema no es principalmente de sobreajuste, sino de **infraajuste**: un árbol de profundidad 3 no dispone de capacidad suficiente para separar ocho tipos de piezas a partir de una representación visual de alta dimensionalidad.

Por tanto, el contraste entre ambas configuraciones muestra que las relaciones existentes entre las características visuales y las clases requieren modelos capaces de representar fronteras de decisión relativamente complejas.

1.4 Track B — Ficha técnica

1.4.1 B.a — Exploración de las variables

Antes de entrenar los modelos se realizó un análisis individual de las once columnas del conjunto tabular.

Table 3: Auditoría inicial de las variables de la ficha técnica.

Variable	Tipo original	Faltantes originales	Faltantes tras reglas
`filename`	object	0	0
`label`	object	0	0
`weight_g`	float	120	120
`height_mm`	float	120	157
`n_studs`	float	192	252
`footprint_mm2`	float	96	96
`price`	object	144	144
`year_released`	float	120	158
`category`	object	120	120
`finish`	object	0	0
`is_defective`	bool	0	0

La variable `filename` es una llave única y, además, contiene información textual que revela directamente la clase, por lo que no puede utilizarse como predictor. Del mismo modo, `label` corresponde a la variable objetivo.

Las variables físicas presentan diferentes problemas de calidad. `weight_g` contiene 120 valores faltantes y alcanza valores de hasta 17,10 g, alejados de su mediana global. `height_mm` contiene también 120 faltantes inicialmente y algunos registros de hasta 96 mm, incompatibles con las dimensiones nominales de las piezas estudiadas.

`n_studs` presenta 192 valores ausentes y contiene valores de -1, físicamente imposibles. `footprint_mm2` presenta 96 valores faltantes. `price` contiene 144 faltantes y está almacenado originalmente como texto, con símbolos monetarios y diferentes separadores decimales.

En `year_released` aparecen años como 0 o 3000, que no representan fechas válidas para este conjunto. Finalmente, `category` contiene 120 faltantes y 20 valores diferentes. Esta última variable está especialmente relacionada con la forma de la pieza y, por tanto, puede actuar como una pista muy cercana a la clase objetivo.

Las variables `finish` e `is_defective` no contienen faltantes, aunque conceptualmente parecen describir más el estado del elemento almacenado que su geometría.

El análisis de correlaciones muestra además una asociación positiva entre peso, número de studs, superficie y precio, por lo que existe cierta redundancia entre las variables físicas.

La Figura 4 permite ver tanto los valores faltantes como estas relaciones entre variables. Se observa que varias columnas presentan una cantidad relevante de datos ausentes y que algunas variables físicas están bastante correlacionadas entre sí.

Conclusión. La ficha técnica contiene información útil para la clasificación, pero no puede utilizarse directamente sin una etapa previa de limpieza. Además, algunas columnas contienen información potencialmente problemática o demasiado próxima al objetivo, por lo que la selección de variables resulta fundamental.

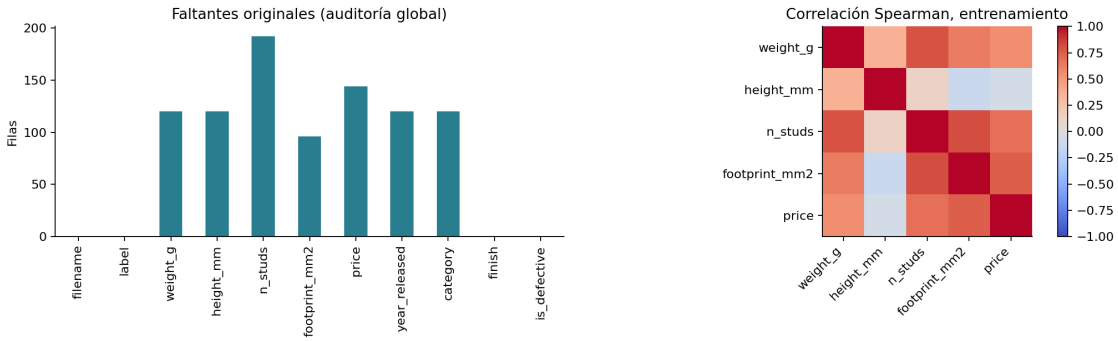


Figure 4: Valores faltantes y relaciones entre variables de la ficha técnica.

1.4.2 B.b — Limpieza y preprocesamiento

El primer paso consistió en convertir `price` a formato numérico, eliminando el símbolo monetario y normalizando los separadores decimales.

Posteriormente se aplicaron reglas explícitas para identificar registros físicamente incompatibles. Los valores no positivos de peso, altura, superficie y precio se consideraron faltantes. También se consideraron inválidos los valores negativos o no enteros de `n_studs`, las alturas superiores a 30 mm y los años situados fuera del intervalo 1932–2026.

Estas reglas incrementan, por ejemplo, los faltantes de `height_mm` desde 120 hasta 157, los de `n_studs` desde 192 hasta 252 y los de `year_released` desde 120 hasta 158.

La Figura 5 muestra cómo cambian las variables después de aplicar las reglas de limpieza. Los nuevos valores faltantes corresponden a datos que se consideraron incompatibles o físicamente imposibles y que posteriormente serán tratados mediante imputación.

En lugar de intentar corregir manualmente valores anómalos, estos se transformaron en ausentes. Posteriormente, la imputación se realizó utilizando la mediana calculada únicamente sobre el conjunto de entrenamiento de cada fold (la mediana es más robusta que la media).

Para el escalado se utilizó `RobustScaler`, basado en la mediana y el rango intercuartílico. Esta elección reduce la influencia de valores extremos sobre el escalado sin necesidad de eliminar observaciones que podrían ser legítimas.

Todo este proceso lo dejé integrado dentro del pipeline de validación. De esta forma, tanto la imputación como los parámetros del escalado se obtienen sin utilizar información de validación ni del conjunto de prueba.

Conclusión. El preprocesamiento permite evitar que errores evidentes de registro se conviertan artificialmente en señales predictivas.

1.4.3 B.c — Selección de variables y análisis de `category`

El modelo principal del Track B utiliza las siguientes variables:

$$\{\texttt{weight_g}, \texttt{height_mm}, \texttt{n_studs}, \texttt{footprint_mm2}, \texttt{price}\}.$$

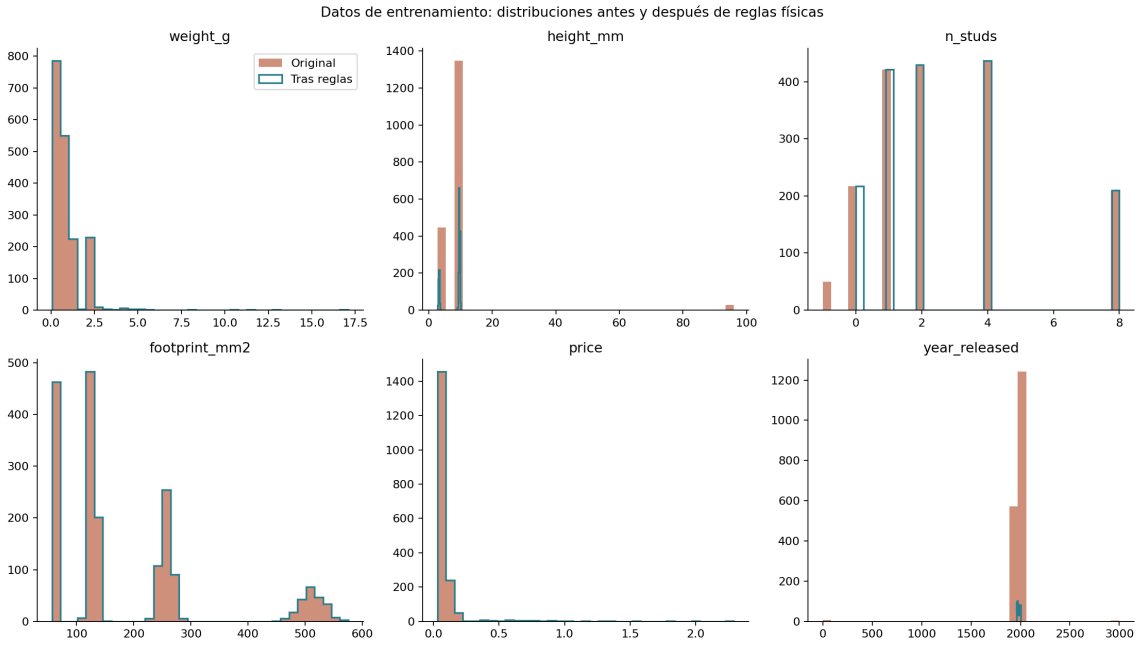


Figure 5: Efecto de las reglas de limpieza sobre las variables tabulares.

Se excluyó `filename` porque contiene directamente el nombre de la pieza, y `label` porque corresponde a la variable objetivo.

También quité `category` del resultado principal. Aunque es una variable muy informativa, asumir que está disponible antes de clasificar la pieza no corresponde al escenario planteado. Para cuantificar su impacto realicé de todos modos un experimento diagnóstico adicional manteniendo fijo el modelo y los hiperparámetros.

Table 4: Efecto de utilizar `category`.

Escenario	**Accuracy CV**	**Accuracy test**
Sin `category`	74,95 %	73,33 %
Con `category`	96,25 %	96,88 %

La incorporación de `category` aumenta la accuracy de validación en 21,30 puntos porcentuales y la accuracy de test en 23,54 puntos.

La magnitud de esta mejora confirma que `category` contiene información extremadamente cercana a la identidad de la pieza. Precisamente por ello, incluirla en el resultado principal generaría una evaluación poco real del problema que nos encontraríamos.

También se comprobó el efecto de añadir `year_released`, `finish` e `is_defective`. La accuracy de validación pasó de 74,95 % a 74,32 %, una reducción de 0,63 puntos porcentuales.

Conclusión. El experimento con `category` demuestra que la ficha podría permitir una clasificación casi perfecta si se dispusiera de información semántica muy relacionada con la clase. Sin embargo, para evaluar correctamente el problema se mantiene fuera del modelo principal. Las variables año, acabado y defecto tampoco aportaron mejoras en esta ejecución.

1.4.4 B.d — Comparación de modelos tabulares

El mejor modelo según validación cruzada es `B_DecisionTree_2`, con profundidad máxima 8 y una accuracy de 74,95 %.

Table 5: Modelos finalistas del Track B.

Modelo	Accuracy CV	Accuracy test
Decision Tree	74,95 %	73,33 %
Random Forest	74,90 %	73,96 %
KNN	71,51 %	70,63 %
SVM	70,89 %	71,04 %

La Figura 6 muestra la comparación entre las doce configuraciones del Track B. Decision Tree y Random Forest presentan los mejores resultados, mientras que las configuraciones de menor rendimiento quedan bastante alejadas de estos dos modelos.

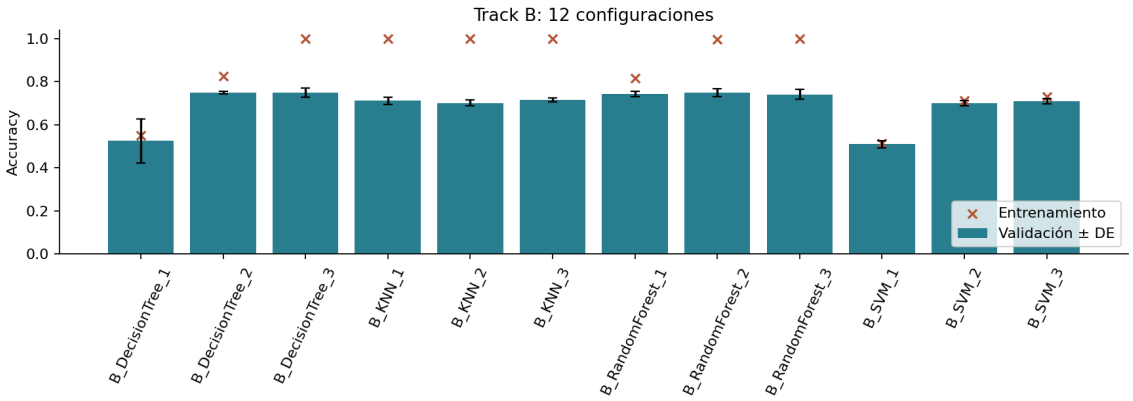


Figure 6: Resultados de validación de las doce configuraciones del Track B.

La diferencia entre Decision Tree y Random Forest en validación es únicamente de 0,05 puntos porcentuales, por lo que ambos presentan un rendimiento prácticamente equivalente. Random Forest obtiene posteriormente un resultado ligeramente superior en test, pero no sería correcto cambiar la selección después de observar el conjunto de prueba.

La peor configuración de B es `B_SVM_1`, con $C = 0,1$, que obtiene únicamente 50,94 % de accuracy de validación.

El análisis de la matriz de confusión del modelo seleccionado permite entender por qué su F1 macro de test, 69,34 %, es inferior a su accuracy de 73,33 %. El modelo no clasifica correctamente ninguna de las 60 piezas `3040 roof tile 1x2`: las 60 son asignadas a la clase `3004 brick 1x2`. Además, aparecen 32 errores de `3005 brick 1x1` hacia `3062 Round Brick 1x1` y 31 errores en la dirección contraria.

Estas tres confusiones concentran 123 de los 128 errores cometidos por el modelo.

Conclusión. La información tabular permite clasificar correctamente una parte importante de las piezas, pero no contiene suficiente información para distinguir determinados pares con dimensiones físicas similares.

Esta limitación explica que el rendimiento del Track B sea claramente inferior al obtenido utilizando las imágenes.

1.5 Track C — Combinación

1.5.1 C.a — Unión y selección de características

La información visual y la ficha técnica se combinaron utilizando `filename` como clave de unión. La unión se realizó mediante el identificador y no mediante la posición de las filas.

El bloque visual inicial contiene un número muy superior de variables al bloque tabular. Para reducir su dimensionalidad utilicé selección univariante mediante ANOVA, conservando el 25 % de las características visuales con mayor asociación con la clase.

Esta selección se ajustó nuevamente dentro de cada fold para evitar utilizar información de validación durante la elección de características.

Al entrenar posteriormente el modelo sobre todo el conjunto de entrenamiento quedaron seleccionadas 443 variables visuales, además de las cinco variables tabulares. Entre las seis características geométricas originales fueron seleccionadas el área relativa y la excentricidad.

Después del escalado, los bloques visual y tabular se ponderan en función de su dimensión. El objetivo es evitar que el bloque visual domine el modelo únicamente porque contiene cientos de variables más que la ficha técnica.

La lógica de la combinación es que ambas fuentes contienen información distinta. HOG representa bordes, contornos y estructura visual, mientras que la ficha aporta información sobre dimensiones y propiedades físicas que podría ayudar a resolver imágenes visualmente ambiguas.

Conclusión. La fusión puede resultar útil si las dos fuentes aportan información complementaria. No obstante, ANOVA realiza una selección univariante y, por tanto, no elimina necesariamente toda la redundancia ni detecta todas las posibles interacciones entre características.

1.5.2 C.b — Comparación de modelos y de los tres tracks

El mejor modelo del Track C por validación es `C_RandomForest_3`, con 200 árboles y sin límite máximo de profundidad. Obtiene una accuracy de validación del 91,04 % y una accuracy de test del 92,29 %.

El peor modelo de C es `C_SVM_1`, que obtiene únicamente un 57,19 % de accuracy en validación.

La Figura 7 permite observar que las configuraciones de Random Forest son las que consiguen los mejores resultados dentro del Track C. También se observa una diferencia clara entre las mejores configuraciones y los modelos con peor rendimiento.

Al comparar los ganadores de los tres tracks sobre exactamente el mismo conjunto de prueba se obtiene:

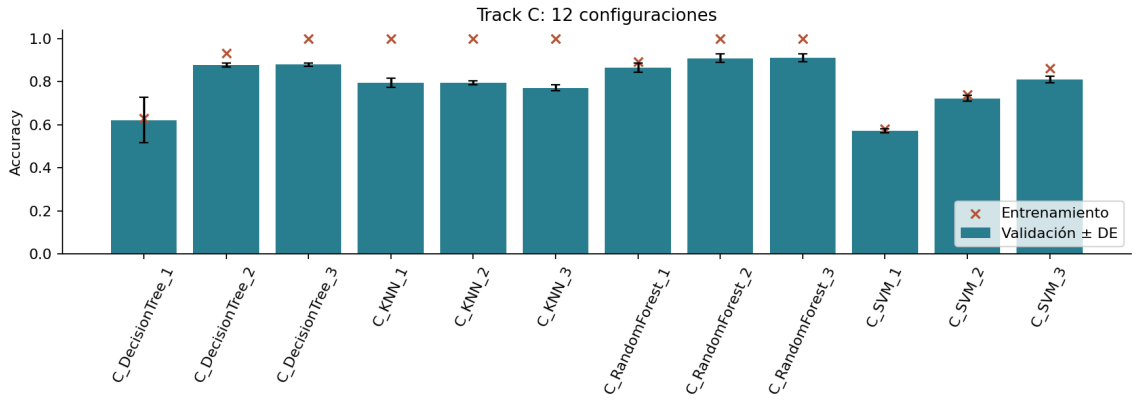


Figure 7: Resultados de validación de las doce configuraciones del Track C.

Table 6: Comparación de los ganadores de A, B y C.

Track	**Modelo**	**Accuracy CV**	**Accuracy test**
A	Random Forest	87,60 %	90,83 %
B	Decision Tree	74,95 %	73,33 %
C	Random Forest	91,04 %	92,29 %

La Figura 8 resume esta comparación. El Track C obtiene el mejor resultado, aunque la diferencia con el Track A es pequeña. En cambio, el Track B queda claramente por debajo de los otros dos enfoques.

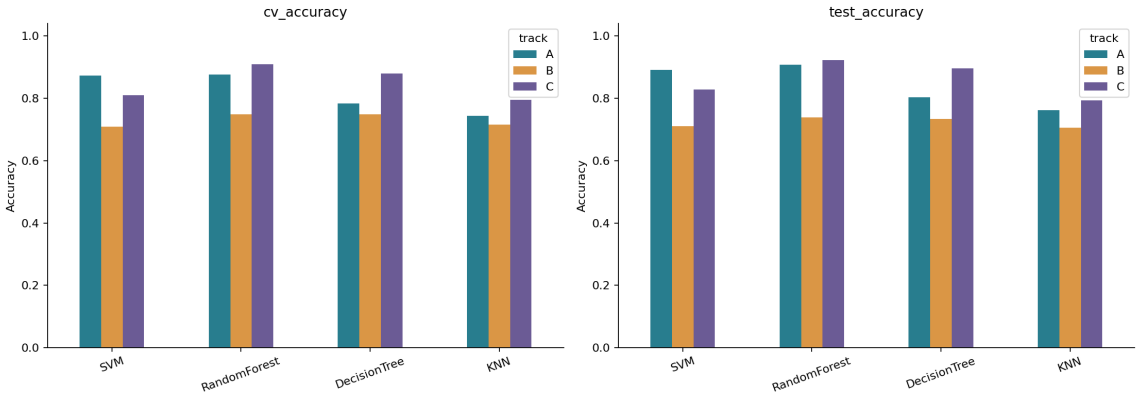


Figure 8: Comparación final de los tres enfoques sobre el mismo conjunto de prueba.

El modelo combinado supera al modelo visual en 3,44 puntos porcentuales de validación y 1,46 puntos en test. Sobre las 480 observaciones de prueba, esto supone pasar de 436 a 443 predicciones correctas, es decir, siete aciertos adicionales.

La diferencia frente al Track B es mucho mayor: el combinado mejora la accuracy de test en 18,96 puntos porcentuales y consigue 91 aciertos adicionales.

Analizando las predicciones individualmente, C corrige 24 imágenes que A clasificaba incorrectamente, aunque también introduce errores en 17 imágenes que A sí clasificaba correctamente. Esto confirma que la mejora

global no significa que el modelo combinado sea superior para todos los ejemplos o todas las clases.

Conclusión. El Track C ofrece el mejor rendimiento global. Sin embargo, la mejora respecto al enfoque puramente visual es relativamente pequeña, mientras que la ventaja respecto a utilizar únicamente la ficha técnica es muy considerable.

1.5.3 C.c — Ranking global por validación

El ranking de las diez mejores configuraciones se determinó exclusivamente utilizando la accuracy obtenida mediante validación cruzada.

Table 7: Top 10 de configuraciones según accuracy de validación.

Puesto	Configuración	Hiperparámetro principal	Accuracy CV
1	C_RandomForest_3	200 árboles, profundidad libre	91,04 %
2	C_RandomForest_2	200 árboles, profundidad 15	90,89 %
3	C_DecisionTree_3	profundidad libre	87,97 %
4	C_DecisionTree_2	profundidad 8	87,66 %
5	A_RandomForest_2	200 árboles, profundidad 15	87,60 %
6	A_RandomForest_3	200 árboles, profundidad libre	87,45 %
7	A_SVM_3	RBF, $C = 10$	87,40 %
8	C_RandomForest_1	200 árboles, profundidad 5	86,56 %
9	A_SVM_2	RBF, $C = 1$	83,49 %
10	C_SVM_3	RBF, $C = 10$	81,04 %

Los tres primeros modelos de este ranking se evaluaron posteriormente sobre el conjunto de prueba:

Table 8: Evaluación en test del top 3 seleccionado mediante validación.

Modelo	Accuracy CV	Accuracy test	Aciertos
C_RandomForest_3	91,04 %	92,29 %	443/480
C_RandomForest_2	90,89 %	93,33 %	448/480
C_DecisionTree_3	87,97 %	89,58 %	430/480

Resulta interesante que `C_RandomForest_2` alcance un 93,33 % en test, superior al 92,29 % de `C_RandomForest_3`. Sin embargo, el segundo había obtenido una accuracy ligeramente inferior durante la validación, 90,89 % frente a 91,04 %.

Por tanto, el ganador oficial debe seguir siendo `C_RandomForest_3`. Cambiar de modelo después de observar que otro obtiene un resultado mejor en test supondría utilizar indirectamente el conjunto de prueba para seleccionar el modelo.

1.5.4 C.d — Matrices de confusión e interpretación

Las matrices de confusión del top 3 muestran que los errores están muy concentrados en determinados pares de piezas.

Para C_RandomForest_3, 28 de sus 37 errores corresponden únicamente a confusiones entre 3004 brick 1x2 y 3040 roof tile 1x2: 17 piezas 3004 se clasifican como 3040, mientras que 11 piezas 3040 se clasifican como 3004.

Table 9: Matriz de confusión de C_RandomForest_3.

	3001	3003	3004	3005	3022	3040	3062	3069
3001	59	1	0	0	0	0	0	0
3003	0	60	0	0	0	0	0	0
3004	0	0	43	0	0	17	0	0
3005	0	0	0	55	0	0	5	0
3022	0	1	0	0	58	0	0	1
3040	0	0	11	0	0	48	0	1
3062	0	0	0	0	0	0	60	0
3069	0	0	0	0	0	0	0	60

El segundo modelo presenta un patrón muy similar:

Table 10: Matriz de confusión de C_RandomForest_2.

	3001	3003	3004	3005	3022	3040	3062	3069
3001	60	0	0	0	0	0	0	0
3003	0	60	0	0	0	0	0	0
3004	0	0	46	0	0	14	0	0
3005	0	0	0	54	0	0	6	0
3022	0	0	0	0	59	0	0	1
3040	0	0	10	0	0	49	0	1
3062	0	0	0	0	0	0	60	0
3069	0	0	0	0	0	0	0	60

Finalmente, el árbol de decisión presenta un mayor número de confusiones:

Table 11: Matriz de confusión de `C_DecisionTree_3`.

	3001	3003	3004	3005	3022	3040	3062	3069
3001	60	0	0	0	0	0	0	0
3003	0	59	0	0	1	0	0	0
3004	0	0	45	0	2	13	0	0
3005	0	0	0	58	0	0	2	0
3022	0	2	1	0	57	0	0	0
3040	0	0	22	0	0	38	0	0
3062	0	0	0	7	0	0	53	0
3069	0	0	0	0	0	0	0	60

Los tres modelos comparten dos focos principales de error. El primero es la confusión entre `3004 brick 1x2` y `3040 roof tile 1x2`. Aunque estructuralmente son piezas diferentes, determinadas perspectivas pueden ocultar parcialmente la pendiente de la pieza `3040`, haciendo que su silueta y dimensiones aparentes sean similares a las de `3004`.

El segundo foco corresponde a `3005 brick 1x1` y `3062 Round Brick 1x1`. Ambas piezas tienen dimensiones generales parecidas, y en determinadas vistas la forma circular de la segunda puede resultar menos evidente.

Esto no permite afirmar exactamente qué regiones de cada imagen utiliza el modelo para decidir, pero sí muestra que sus errores no son aleatorios: se concentran en clases con características visuales y físicas similares.

1.6 Conclusiones

Los experimentos permiten extraer varias conclusiones principales.

En primer lugar, la información visual es considerablemente más discriminativa que la ficha técnica utilizada de manera aislada. El mejor modelo visual alcanza un 90,83 % de accuracy en test, frente al 73,33 % del modelo seleccionado en el Track B. La principal limitación de la ficha técnica aparece en pares de piezas que comparten dimensiones y propiedades físicas similares, para los cuales faltan variables capaces de representar diferencias de forma.

En segundo lugar, combinar ambas fuentes produce el mejor resultado global. El modelo `C_RandomForest_3` alcanza un 92,29 % de accuracy en test y clasifica correctamente 443 de las 480 piezas. Esto supone siete aciertos adicionales respecto al mejor modelo visual y 91 respecto al ganador tabular.

Sin embargo, la mejora del Track C frente al Track A es únicamente de 1,46 puntos porcentuales en test. Por tanto, no puede afirmarse que la combinación sea siempre la opción más conveniente. Si la ficha técnica ya está disponible y mantenerla no supone un coste significativo, el Track C sería la elección preferible porque ofrece el mejor rendimiento. Si, por el contrario, obtener, limpiar y mantener los datos tabulares añade complejidad al sistema, el Track A resulta una alternativa muy competitiva, ya que consigue un rendimiento apenas inferior utilizando únicamente imágenes.

Además, no puede atribuirse toda la mejora de C exclusivamente a la incorporación de la ficha técnica. En el enfoque combinado también se aplica selección ANOVA y una ponderación específica de los bloques de características. Para aislar completamente el efecto de añadir información tabular sería necesario realizar un experimento de ablación en el que A y C compartieran exactamente el mismo preprocesamiento visual.

Otro resultado relevante es que una accuracy global elevada no implica que todas las clases se resuelvan con la misma facilidad. Incluso el mejor modelo concentra gran parte de sus errores en las confusiones 3004–3040 y 3005–3062. Esto indica que futuras mejoras deberían dirigirse específicamente a incorporar características capaces de distinguir estos pares.

Finalmente, los resultados corresponden a una única colección de renders de ocho clases LEGO. Por ello, las cifras obtenidas no permiten garantizar el mismo rendimiento ante fotografías reales, cambios fuertes de iluminación, fondos diferentes, nuevas orientaciones o clases no presentes durante el entrenamiento.

En conjunto, el **Track C ofrece el mejor rendimiento predictivo**, mientras que el **Track A presenta probablemente la mejor relación entre rendimiento y simplicidad**. El Track B demuestra que las variables tabulares contienen información relevante, pero por sí solas no son suficientes para resolver de forma robusta las diferencias geométricas entre todas las piezas.

References

[1] N. Dalal y B. Triggs, "Histograms of Oriented Gradients for Human Detection," *Proceedings of CVPR*, 2005. https://doi.org/10.1109/CVPR.2005.177

[2] scikit-image developers, "Histogram of Oriented Gradients." https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_hog.html

[3] scikit-image developers, "skimage.measure: region properties and perimeter." https://scikit-image.org/docs/stable/api/skimage.measure.html

[4] scikit-learn developers, "Preprocessing data." https://scikit-learn.org/stable/modules/preprocessing.html

[5] scikit-learn developers, "Common pitfalls and recommended practices." https://scikit-learn.org/stable/common_pitfalls.html

[6] scikit-learn developers, "Feature selection." https://scikit-learn.org/stable/modules/feature_selection.html

[7] scikit-learn developers, "Support Vector Machines." https://scikit-learn.org/stable/modules/svm.html

[8] scikit-learn developers, "Decision Trees." https://scikit-learn.org/stable/modules/tree.html

[9] scikit-learn developers, "Ensembles: Random Forests." https://scikit-learn.org/stable/modules/ensemble.html

[10] scikit-learn developers, "Nearest Neighbors." https://scikit-learn.org/stable/modules/neighbors.html

[11] J. Hazelzet, "Images of LEGO Bricks," Kaggle. https://www.kaggle.com/datasets/joosthazelzet/lego-brick-images

[12] IIC2613 Inteligencia Artificial, *Tarea 1: Aprendizaje Automático y Reforzado*, Pontificia Universidad Católica de Chile, 2026-2. Material proporcionado por el curso.

1.7 Anexo de IA

Para la realización de esta parte del trabajo se utilizó inteligencia artificial generativa como herramienta de apoyo en la planificación del análisis, revisión y documentación del código y la interpretación inicial de los resultados. Los resultados numéricos presentados en esta sección proceden, en todos los casos, de la ejecución del notebook entregado. La selección de modelos y las conclusiones finales se realizaron a partir de dichos resultados, evitando utilizar el conjunto de prueba para escoger hiperparámetros.

- **Herramientas de IA utilizadas:** ChatGpt fue utilizado para la generación del código correspondiente
- **Finalidad de su uso:** Para agilizar toda la parte de programar los experimentos, el pipelines de esta parte del proyecto así como la generación de los gráficos y tablas que posteriormente serían analizadas. También se ha usado para redactar las referencias.
- **Prompts relevantes:**
  - Analiza el repo entero dónde está el enunciado, los códigos y los datasets proporcionados por el profesor. Así planteamos como encarar el proyecto. Antes de crear código quiero planificar como avanzaremos esta primera parte (la de las piezas).
  - Tengo que ejecutarlo en colab porque tengo GPU pero si me generas los códigos necesarios ya los muevo yo a colab. En caso de que haya alguna decisión que tomar respecto al código preguntame antes de tomar tu la decisión
- **Respuestas relevantes:** El código utilizado en el ipynb de esta primera parte (adjunto aparte) donde están los resultados analizados.

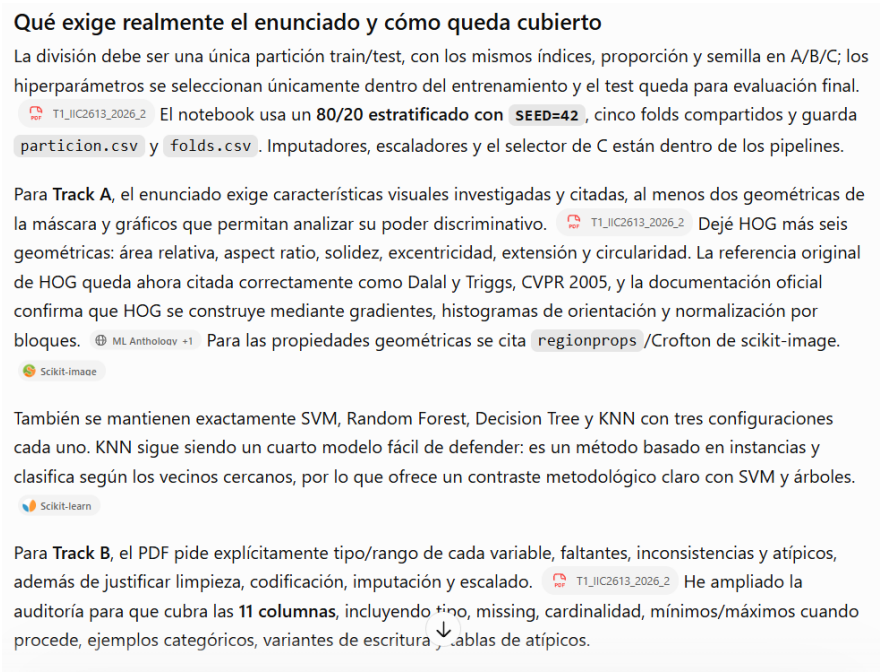


Figure 9: Respuesta de ChatGPT

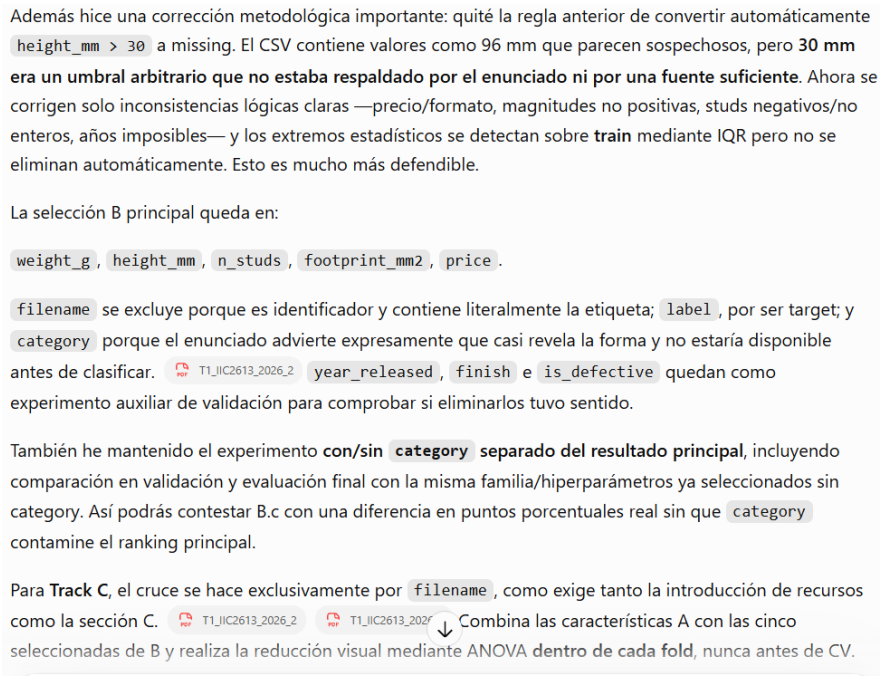


Figure 10: Respuesta de ChatGPT